Name : Dasari Joshua Sonal USN : 01FE24BCI078 Div : A
Roll no : 148

Architecture	High-level structure	Advantages	Limitations
Modular Monolith Architecture	One single app with code neatly organized into separate compartments (Wallet, Payment, Cashback).	• Simple to launch and manage • Clean code without extra server costs	Runs as one big application • Compartments can get messy if rules aren't followed
Microservices Architecture	System is split into tiny, completely independent mini-apps connected over the internet.	•Update or scale one feature without touching the rest • If Cashback crashes, Payments still work	• Very complex to set up and manage • Network delays between mini-apps
Event driven Architecture	Apps talk by shouting news updates (e.g., "Payment Done!") that other parts listen for	• Fast checkout for users • Tasks like cashback happen instantly in the background	• Harder to trace and fix errors • Must handle duplicate or delayed messages carefully

PART-A

Criterion	Architecture A	Architecture B	Architecture C
Maintainability	High	High	Medium
Scalability	Medium	High	High

Criterion	Architecture A	Architecture B	Architecture C
Ease of testing	High	Low	Medium
Database access control	Medium	High	High
Authentication placement	Medium	Medium	Medium
Suitability for FinPay requirements	High	Medium	High
Implementation complexity	Low	High	Medium

Architecture	Why it was considered	Selected / Not selected	Reason or limitation for FinPay
Architecture A	Organizes FinPay's core capabilities (Wallet, Payment, Cashback, Reconciliation) into clean internal modules while maintaining a single deployment unit.	Selected	Provides clean code structure and fast development with low infrastructure costs, making it ideal for FinPay's core foundation
Architecture B	Allows independent deployment and isolated scaling for each FinPay operation (Wallet, Payment, Merchant)	Not Selected	Introduces high distributed system complexity, network latency, and operational burden before FinPay's volume strictly requires it.

Architecture	Why it was considered	Selected / Not selected	Reason or limitation for FinPay
Architecture C	Enables instant payment execution while running secondary operations (cashback calculations, reconciliation records) asynchronously.	Selected	Keeps student checkout response times fast and reliable by executing secondary calculations in the background.

PART-B

STRUCTURED PROMPT

ARCHITECTURE PROMPT — FinPay Campus Wallet Platform

You are a software architect. Design the server-side architecture for FinPay using the six-element specification below. Follow all six elements exactly.

[ELEMENT 1 — SYSTEM SCOPE AND TECHNOLOGY STACK]

FinPay campus wallet platform managing student wallets, payment transactions, cashback, merchants, reconciliation, and auth. Tech stack: Python 3.11+, FastAPI, SQLAlchemy 2.x, Pydantic v2, Alembic, and PostgreSQL. Authentication uses JWT with python-jose.

[ELEMENT 2 — LAYER STRUCTURE AND RESPONSIBILITY]

Use a Modular Monolith architecture divided into six business modules (auth, wallet, payment, cashback, merchant, reconciliation). Inside each module, use four layers: Router — HTTP handling; Service — business logic; Repository — database access; Model — data definitions.

[ELEMENT 3 — LAYER ACCESS RULES]

Inside a module, follow: Router → Service → Repository → Model. No reverse calls, no layer skipping. Modules must talk to each other only through public service interfaces or internal events. No direct database access across different modules.

[ELEMENT 4 — CROSS-CUTTING CONCERNS]

Use shared JWT dependencies for auth. Services raise domain exceptions, and Routers convert them into HTTP responses. Repositories catch database errors. Use Pydantic schemas for request and response validation.

[ELEMENT 5 — KEY ENTITY / COMPONENT PLACEMENT]

Place User, Wallet, Transaction, Merchant, Cashback, and Reconciliation models inside their module models/ folders. Use the main services: auth_service.py, wallet_service.py, payment_service.py, cashback_service.py, merchant_service.py, and reconciliation_service.py.

[ELEMENT 6 — EXPLICIT EXCLUSIONS]

Do not put business logic in routers. Do not make direct database calls in services. Do not calculate cashback outside cashback_service.py. Do not modify wallet balances outside wallet_service.py. Do not put authentication logic in domain services or models. Do not define models outside models/.

Generate ONLY:
Architecture style
High-level architecture diagram
Folder/component structure
Layer/component responsibilities
Dependency flow
Key entity placement
Authentication and error-handling structure
Important assumptions, if any

DO NOT generate Python code, function implementations, SQL queries, test cases or complete application files.

PART-C

Aspect	Architecture selected in Part A	AI-generated architecture	Difference / Observation
Architecture structure	A single deployable application internally divided into isolated, well-defined business modules.	Multiple small, independently deployable services distributed across separate servers.	Selected architecture deploys as one single unit, while AI-generated architecture requires deploying and managing multiple server units
Main components / layers	6 internal modules (Auth, Wallet, Payment, Cashback, Merchant, Reconciliation) sharing one database	API Gateway, standalone microservices (Wallet Service, Payment Service, etc.), and dedicated database per service	Selected architecture avoids API Gateway and multi-database setup costs while maintaining clear feature separation

Aspect	Architecture selected in Part A	AI-generated architecture	Difference / Observation
Responsibilities	Modules handle domain logic internally; money transfers execute within local ACID database transactions.	Services manage logic independently; requires complex distributed transactions across multiple databases	elected architecture guarantees financial data consistency without needing complex distributed Saga patterns
Interaction / communication	Fast in-process method calls and internal event dispatchers within the same application memory.	Network calls (REST/gRPC over HTTP) and external message brokers (Kafka / RabbitMQ)	Selected architecture eliminates network latency and inter-service security checks during payment checkout.
Overall difference	—	—	Modular Monolith is superior for FinPay. It delivers strict code organization and high maintainability without the heavy infrastructure costs, network lag, and operational complexity of AI-generated Microservices.

PART-D

D2. Write ADR-001

Field	Content / Requirements
ID and Title	**ADR-001:** Selection of Modular Monolith Architecture for FinPay Campus Wallet Platform
Status	Accepted
Context	FinPay needs a reliable server-side system to handle student wallets, payment processing, cashback rewards, merchant management, monthly reconciliation, and authentication. The system must enforce clear business boundaries so future feature additions stay organized, while avoiding high server costs and distributed system errors during payment processing.
Decision	Adopt a **Modular Monolith Architecture** divided into six strict internal business modules (auth, wallet, payment, cashback, merchant, reconciliation) deployed as a single application unit.
Consequences	**• Easier: Keeps setup, testing, and deployment simple while guaranteeing fast, local database transactions across wallet updates and payment records.** **• Harder: Requires strict code reviews and discipline to stop developers from creating shortcuts between modules over time.** **• Impossible: Prevents scaling or deploying individual modules on separate server instances independently without first refactoring them into microservices**
Review Trigger	Reconsider this decision if FinPay's transaction traffic exceeds single-database capacity or if specific modules (like Payment Processing) require independent server scaling.

PART-E

RR Section	What to record
1. Architecture version	• **System Name:** FinPay Campus Wallet Platform • **Version:** v1.0.0 • **Generation Date:** September 25, 2026 • **Prompt Reference:** PROMPT-FINPAY-MODULAR-MONOLITH-V1

2. Architecture prompt	**ARCHITECTURE PROMPT — FinPay Campus Wallet Platform** ou are a software architect. Design the server-side architecture for FinPay using the six-element specification below. Follow all six elements exactly. **[ELEMENT 1 — SYSTEM SCOPE AND TECHNOLOGY STACK]** FinPay campus wallet platform managing student wallets, payment transactions, cashback, merchants, reconciliation, and auth. Tech stack: Python 3.11+, FastAPI, SQLAlchemy 2.x, Pydantic v2, Alembic, and PostgreSQL. Authentication uses JWT with python-jose. **[ELEMENT 2 — LAYER STRUCTURE AND RESPONSIBILITY]** Use a Modular Monolith architecture divided into six business modules (auth, wallet, payment, cashback, merchant, reconciliation). Inside each module, use four layers: Router — HTTP handling; Service — business logic; Repository — database access; Model — data definitions. **[ELEMENT 3 — LAYER ACCESS RULES]** Inside a module, follow: Router → Service → Repository → Model. No reverse calls, no layer skipping. Modules must talk to each other only through public service interfaces or internal events. No direct database access across different modules. **[ELEMENT 4 — CROSS-CUTTING CONCERNS]** Use shared JWT dependencies for auth. Services raise domain exceptions, and Routers convert them into HTTP responses. Repositories catch database errors. Use Pydantic schemas for request and response validation. **[ELEMENT 5 — KEY ENTITY / COMPONENT PLACEMENT]** Place User, Wallet, Transaction, Merchant, Cashback, and Reconciliation models inside their module models/ folders. Use the main services: auth_service.py, wallet_service.py, payment_service.py, cashback_service.py, merchant_service.py, and reconciliation_service.py. **[ELEMENT 6 — EXPLICIT EXCLUSIONS]**

RR Section	What to record
	Do not put business logic in routers. Do not make direct database calls in services. Do not calculate cashback outside cashback_service.py. Do not modify wallet balances outside wallet_service.py. Do not put authentication logic in domain services or models. Do not define models outside models/. Generate ONLY: 1. Architecture style 2. High-level architecture diagram 3. Folder/component structure 4. Layer/component responsibilities 5. Dependency flow 6. Key entity placement 7. Authentication and error-handling structure 8. Important assumptions, if any.
3. Generated architecture	• **Style:** Modular Monolith Architecture • **Modules:** auth_module/, wallet_module/, payment_module/, cashback_module/, merchant_module/, reconciliation_module/ • **Internal Layer Structure:** Router → Service → Repository → Model • **Responsibilities:** Clear business boundaries with User, Wallet, Transaction, Merchant, Cashback, and Reconciliation entities localized inside their respective module folders. • **Key Constraints:** Strict public service boundaries, no direct cross-module SQL queries, event-driven internal dispatching for background tasks, and centralized JWT authentication checks
4. Decision	**APPROVED** • **Timestamp:** September 25, 2026 — 04:45 PM • **Reviewer Name:** Software Architecture Review Committee / Lead Architect